

TopV: Compatible Token Pruning with Inference Time Optimization for Fast and Low-Memory Multimodal Vision Language Model

Cheng Yang^{1*}, Yang Sui^{2*†}, Jinqi Xiao¹, Lingyi Huang¹, Yu Gong¹, Chendi Li³,
 Jinghua Yan³, Yu Bai⁴, Ponnuswamy Sadayappan³, Xia Hu², Bo Yuan¹

¹Rutgers University, ²Rice University, ³The University of Utah, ⁴California State University, Fullerton
 cheng.yang@rutgers.edu, yang.sui@rice.edu, bo.yuan@soe.rutgers.edu

Abstract

*Vision-Language Models (VLMs) demand substantial computational resources during inference, largely due to the extensive visual input tokens for representing visual information. Previous studies have noted that visual tokens tend to receive less attention than text tokens, suggesting their lower importance during inference and potential for pruning. However, their methods encounter several challenges: reliance on greedy heuristic criteria for token importance and incompatibility with FlashAttention and KV cache. To address these issues, we introduce **TopV**, a compatible **T**oken **P**runing with inference **T**ime **O**ptimization for fast and low-memory VLM, achieving efficient pruning without additional training or fine-tuning. Instead of relying on attention scores, we formulate token pruning as an optimization problem, accurately identifying important visual tokens while remaining compatible with FlashAttention. Additionally, since we only perform this pruning once during the prefilling stage, it effectively reduces KV cache size. Our optimization framework incorporates a visual-aware cost function considering factors such as Feature Similarity, Relative Spatial Distance, and Absolute Central Distance, to measure the importance of each source visual token, enabling effective pruning of low-importance tokens. Extensive experiments demonstrate that our method outperforms previous token pruning methods, validating the effectiveness and efficiency of our approach.*

1. Introduction

Vision-Language Models (VLMs) have significantly advanced the interpretation and synthesis of information across textual and visual modalities with their cutting-edge techniques. VLMs process a combination of visual inputs and textual data, such as an image accompanied by instruc-

tional text, to generate textual outputs like descriptions or answers to visual queries, facilitating a range of applications, including visual recognition [35, 49, 56, 61], image captioning [3, 10, 36, 63], visual question answering [2, 16, 33, 53], and optical character recognition (OCR) [13, 20, 38, 59] tasks.

Despite their versatility, VLMs require significant computational resources and substantial memory during inference. In addition to processing text tokens (e.g., system tokens, instruction tokens), VLMs need to handle a considerable volume of visual tokens, which substantially increase both computational costs and memory usage. For example, in the OCR task, LLaVA-v1.5 [28] translates an image into 576 visual tokens, constituting up to 87% of the total input sequences. Similarly, InternVL [6, 7, 14], depending on image resolution, produces between 256 and 1792 visual tokens, accounting for up to 95% of the total tokens. These inefficiencies present a serious obstacle to scaling VLMs in resourced-limited environments.

Recently, observed by work [5], visual tokens often receive less attention compared to text tokens, indicating their lower importance during the VLM inference. Consequently, visual token pruning [5, 25, 47] is proposed to reduce redundant visual information. FastV [5] dynamically prunes unimportant visual tokens based on their received attention score. LLaVA-PruMerge [42] retains the most critical visual tokens and then merges less important tokens into them. VTW [25] prunes all vision tokens from certain layers. However, they face three drawbacks:

- **Greedy Heuristic Criteria for Token Importance.** Previous works such as FastV [5], and LLaVa-PruMerge [42] primarily rely on attention scores to measure the importance, assuming higher scores indicate greater relevance. In Contrast, VTW [25] prunes all visual tokens after certain layers without considering importance metrics. As a result, these methods fail to accurately capture each token’s contribution to the VLM model.
- **Incompatibility with FlashAttention.** FlashAttention [9] computes output tokens by fusing the softmax with the

*Equal Contribution.

†Corresponding Author.

Value matrix multiplication that reduces memory access costs and accelerates training/inference. However, previous works FastV [5] and LLaVa-PruMerge [42] utilize the attention scores for token importance, requiring explicit computation and negating FlashAttention’s efficiency benefits.

- **Incompatibility with KV Cache.** Previous works such as FastV [5] dynamically prune tokens at each decoding step, requiring variable KV cache sizes. As a result, the full KV cache from the prefilling stage must be stored, negating memory savings.

To address above issues within a token pruning framework, we propose TopV, an optimization-based token pruning method for efficient VLM inference, compatible with KV cache and FlashAttention [9] without extra training. Instead of using attention scores, we optimize token selection based on their contribution to subsequent layers, ensuring accurate importance estimation. Our approach avoids additional attention computations, leveraging FlashAttention fully. Furthermore, we prune tokens once during prefilling, maintaining a reduced set throughout decoding, effectively minimizing KV cache size and memory usage. Our contributions are summarized as follows:

- We propose a training-free token reduction method for VLMs that optimizes pruning during the inference prefilling stage, reducing computation while maintaining compatibility with FlashAttention and KV cache. By pruning once and avoiding explicit attention score calculations, our approach enables a fast, low-memory VLM.
- We formulate the optimization problem by incorporating vision-specific factors such as feature similarity, spatial relationships, and central distance to enhance token selection and the visual-aware cost function. Token importance is then computed using the Sinkhorn algorithm, followed by a token recovery strategy after pruning.
- We conduct extensive experiments on LLaVA and InternVL2 models across multiple tasks, demonstrating that our method surpasses state-of-the-art token pruning approaches. Specifically, on InternVL2 models, it improves accuracy by 1.2%, reduces visual FLOPs by 47%, lowers dynamic memory usage by 61%, and achieves 2.1× inference efficiency. For LLaVA models, it improves performance by 0.39%, reduces visual FLOPs by 50%, lowers dynamic memory usage by 49%, and achieves 1.68× inference efficiency compared to previous methods.

2. Related Work

Vision-Language Models. Recent advances in Vision-Language Models (VLMs) have made significant strides in integrating vision, video, and language modalities, with models like CLIP [41], LLaVA [26, 28], and BLIP-2 [19] enabling tasks such as image captioning and zero-shot image-to-text generation. Video-LLaMA [58] and LLaVA-

NeXT [27] expanded these capabilities to include video and audio processing, further enhancing multimodal understanding. However, as models scale to higher resolutions or multiple video frames, they require an exponentially growing number of tokens, leading to substantial computational challenges. For instance, LLaVA processes high-resolution images into thousands of tokens, and video models like VideoLLaVA [23] allocate even more to represent multiple frames. This growing token demand presents a significant bottleneck, highlighting the need for more efficient computational strategies to fully realize the potential of VLMs.

Visual Token Pruning for VLMs. Visual tokens often vastly outnumber text tokens in VLMs. In the previous works, vision token pruning can be divided into two primary categories: pruning before the LLM and pruning within the LLM. In the case of pruning before the LLM, visual tokens are discarded before entering the model, thereby reducing the computational burden on the LLM. For instance, LLaMA-VID [22] employs a Q-Former [19] to compress visual tokens with the assistance of context tokens before passing them to the LLM. Similarly, DeCo [55] utilizes adaptive pooling to downsample visual tokens before they are entered into the LLM. Both methods focus on early token reduction, improving efficiency by keeping only the most relevant visual information. For pruning within the LLM, the model can consider token interactions and make pruning decisions based on more informed features in layers of LLM. For example, LLavaVolta [4] proposes multi-stage training to gradually reduce the number of visual tokens, though achieving this in a train-free manner remains challenging. FastV [5] predicts the importance of visual tokens in the early layers of the LLM and prunes tokens accordingly. However, FastV relies on dynamic pruning and explicit calculation of attention maps, which can complicate KV cache management and FlashAttention. Finally, VTW [25] employs aggressively pruning visual tokens in the middle layers of the LLM. While this approach works well for recognition tasks, it is less effective for tasks like OCR, caption generation, or VQA, where more detailed visual information is required.

3. Method

In this section, we introduce our token pruning method. We begin with formulating the visual token pruning in VLMs as an optimization problem. Then, we detail the essential components required to develop and solve this problem. Finally, we present our proposed recovery-based pruning strategy.

3.1. Token Importance Formulation

Obtaining the importance of each visual token can be formulated as an optimization problem. Given the input tokens in the L_i layer, token pruning aims to keep tokens that have the greatest impact on subsequent layers, while removing

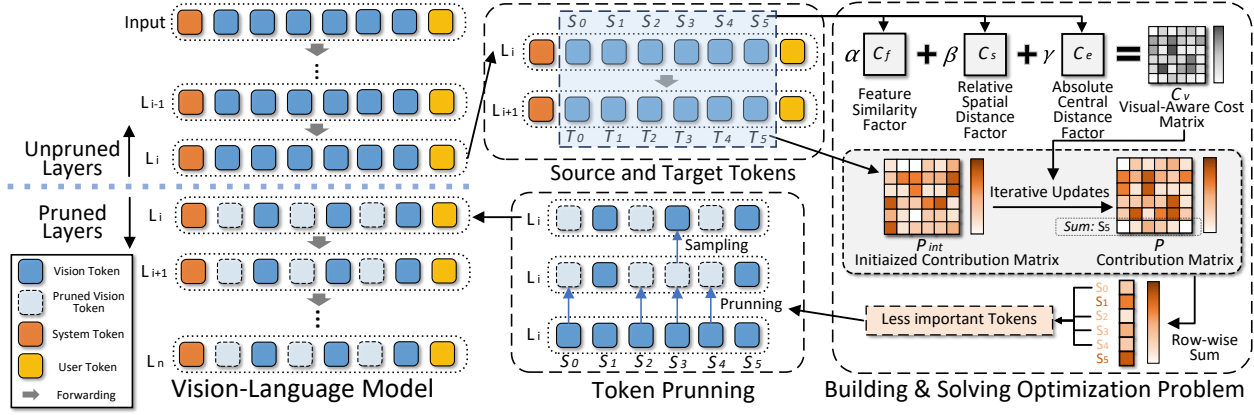


Figure 1. The pipeline of proposed TopV. In the prefilling stage during the inference time, we aim to prune the visual tokens in the L_i layer. First, we select input visual tokens as source tokens and then collect output tokens after the Post-LN layer within the same layer as target tokens. We then formulate the token pruning as an optimization problem, incorporating factors such as feature similarity, relative spatial distance, and absolute central distance. Using the Sinkhorn algorithm to solve this problem, we obtain the contribution matrix of source tokens and their importance. Based on these values, we prune unimportant tokens and uniformly recover a subset of pruned tokens to maintain structural integrity. Starting from the layer L_{i+1} , tokens are consistently pruned, leading to faster, low-memory VLM inference. Following [5], we set $L_i = 2$ in our experiments. Notably, our optimization process requires only 2 ms, comprising less than 1% of total inference time.

those with minimal influence. As shown in Fig. 1, we define the input visual tokens as source tokens $\mathcal{S} = \{s_i \in \mathbb{R}^d \mid i = 1, 2, \dots, N\}$, while the output tokens from this layer are viewed as target tokens $\mathcal{T} = \{t_j \in \mathbb{R}^d \mid j = 1, 2, \dots, N\}$, where d represents the dimensionality of each token. The objective is to identify the important source tokens $\mathcal{S}$ that most significantly contribute to constructing the target tokens. Inspired by Optimal Transport [15, 37, 39, 48], the contribution of each source token can be obtained by solving this Optimal Transport optimization problem:

$$\mathbf{P}^* = \arg \min_{\mathbf{P}} \left(\sum_{i=1}^N \sum_{j=1}^N \mathbf{P}_{i,j} \mathbf{C}_v(s_i, t_j) \right), \quad (1)$$

where $\mathbf{P}$ represents the transport plan. $\mathbf{P}^*$ denotes the optimal transport plan, also referred to as the *Contribution Matrix*, reflecting the contribution scores of the source tokens in constructing the target tokens. $\mathbf{C}_v(\cdot)$ is the cost function representing the cost of transporting mass from $\mathcal{S}$ to $\mathcal{T}$. After obtaining the $\mathbf{P}^*$, tokens with lower contributions can be identified and discarded to reduce computational cost and memory usage. Note that we follow [5] to measure the importance of visual tokens at $L_i = 2$ layer and remove the corresponding tokens within layers starting from it in our experiments.

Formulating and solving Eq. (1) requires several key components. First, in Sec. 3.2, we introduce a token analysis to determine the position of the input tokens $\mathcal{S}$ and target tokens $\mathcal{T}$ within the layer. Second, in Sec. 3.3, we establish a visual-aware cost function $\mathbf{C}_v$ by incorporating the visual properties of the tokens in VLMs. Third, in Sec. 3.4, we

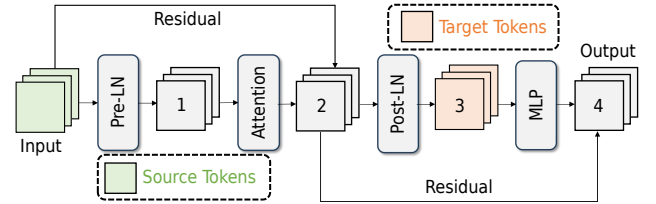


Figure 2. Illustration of target token positions within a transformer layer of VLMs. Positions 1, 2, 3, and 4 correspond to the outputs from the Pre-LN layer, Attention layer, Post-LN layer, and MLP layer, respectively. We empirically select the output after the Post-LN layer (Position 3) as the target tokens.

iteratively solve Eq. (1) to obtain the *Contribution Matrix* $\mathbf{P}^*$, and then unimportant tokens are able to be pruned according to their contribution scores. Finally, in Sec. 3.5, we employ a token recovery strategy to support and complement the collapsed visual tokens.

3.2. Determining the Target Tokens

Given the to-be-pruned visual tokens as source tokens $\mathcal{S}$, we analyze to determine the appropriate position of target tokens $\mathcal{T}$. Fig. 2 illustrates a typical transformer block for LLMs within VLMs, where potential options for selecting target tokens include the outputs of: (1) Pre-LN, (2) Multi-Head Attention, (3) Post-LN, and (4) MLP.

First, since the core information processing primarily occurs in the Attention module, the Pre-LN output does not effectively capture the full function of the transformer block. Second, outputs after the Attention and MLP modules pass

through the Residual module. The similarity induced by such residual connections can obscure distinctions between source and target tokens. By adopting the Post-LN output, we address this issue, ensuring that the target tokens reflect meaningful distinctions from the source tokens while also capturing the primary function of the Attention module.

Consequently, we select the output of (3) Post-LN as our target tokens. In Sec. 4.3, we present ablation studies evaluating various target token positions, with empirical results supporting our analysis.

3.3. Visual-Aware Cost Function

In Eq. (1), the cost function is defined as $C_v(\mathcal{S}, \mathcal{T}) \in \mathbb{R}^{N \times N}$ where each element $C_v(s_i, t_j)$ quantifies the cost of moving a unit of mass from $s_i \in \mathcal{S}$ to $t_j \in \mathcal{T}$, where $i = 1, 2, \dots, N, j = 1, 2, \dots, N$. In the following section, we simply adopt $C_v(s_i, t_j)$ to illustrate the proposed factors.

Feature Similarity Factor. Transforming similar tokens requires minimal adjustment, which lowers the overall cost. Therefore, we incorporate the similarity between the source token $s_i \in \mathcal{S}$ and target token $t_j \in \mathcal{T}$ as the basic cost function. The similarity factor assigns a lower cost to pairs of tokens that are close in feature space, encouraging the optimal transport solution to align similar source and target tokens. Conversely, dissimilar pairs receive a higher cost, effectively penalizing less relevant matches. This similarity-based cost is effectively measured using the L2 norm:

$$C_f(s_i, t_j) = \|s_i - t_j\|_F^2, \quad (2)$$

where each entry $C_f(s_i, t_j)$ represents the transformation cost between a specific pair of tokens (s_i, t_j) .

Relative Spatial Distance Factor. In vision-related models (e.g., ViT, VLMs), considering spatial relationships among visual tokens is crucial for accurately capturing each token’s relative location and the overall structural layout within an image. For instance, positional encodings [43] is commonly used in CLIP [41], ViT [11] and VLMs [42], where positional information is encoded through transformations of token embeddings. These position encodings allow the tokens to maintain spatial correspondence with the original image. Generally, in vision tasks, the correlation between neighboring tokens is stronger than between tokens that are far apart, resulting in lower costs for transformations. To effectively leverage this spatial awareness, we introduce a Relative Spatial Distance Factor into the cost function. Specifically, we define the transport cost between spatially proximate tokens (i.e., tokens representing the same or neighboring regions within the image) to be lower than the cost for tokens representing distant regions. This spatial distance

factor is defined by the relative Gaussian distance:

$$C_s(s_i, t_j) = 1 - \exp\left(-\frac{(x_{s_i} - x_{t_j})^2 + (y_{s_i} - y_{t_j})^2}{2\sigma^2}\right), \quad (3)$$

where (x_{s_i}, y_{s_i}) and (x_{t_j}, y_{t_j}) represent the positions indices of the source token s_i and the target token t_j . For the LLaVA-v1.5-7B model, which has 576 vision tokens, the original image is divided into 24×24 patches, meaning that the positional indices $x, y \in \{0, 1, \dots, 23\}$. We illustrate the Gaussian distance from a source token s_i to a target token t_j used in relative spatial distance $C_e(s_i, t_j)$.

Absolute Central Distance Factor. Central parts of images typically carry the most relevant visual information for tasks such as object detection and scene understanding. Transforming source tokens from less informative areas, such as the edges of an image, into other areas demands extra information, leading to a higher cost. Accordingly, we introduce the Absolute Central Distance Factor into the cost function:

$$C_e(s_i, t_j) = \sqrt{(x_{s_i} - x_c)^2 + (y_{s_i} - y_c)^2}, \quad (4)$$

where c_x and c_y denote the coordinates of the center token in the original image. Specifically, $x_c = x_y = \sqrt{N}/2$, and $x, y \in \{0, 1, \dots, \sqrt{N}\}$. Note that we only consider this factor for source tokens. Therefore, for a given source token s_i , the values of $C_e(s_i, t_j)$ remain constant for all $j = 1, 2, \dots, \sqrt{N}$. we depict the Euclidean distance from a source token s_i to center coordinates used in the absolute central distance $C_e(s_i, t_j)$.

Overall Visual-Aware Cost Function. We normalize each of the three factors to a range of $[0, 1]$ to avoid extreme imbalances among the terms. Then, taking into account these normalized factors, the visual-aware cost function is formulated as follows:

$$C_v(s_i, t_j) = \alpha C_f(s_i, t_j) + \beta C_s(s_i, t_j) + \gamma C_e(s_i, t_j), \quad (5)$$

where α, β, γ serve as hyperparameters that adjust the relative weight of each cost factor. We assign a smaller value to γ compared to α and β to prevent excessive pruning of tokens at the image edges. More ablation studies are discussed in Sec. 4.3.

3.4. Obtaining Token Importance

Solving the Optimization Problem. To solve Eq. (1), we employ the Sinkhorn algorithm [8, 18, 40] to iteratively solve the optimization problem as shown in Alg. 1. p and q are the normalized distributions over the source tokens $\mathcal{S}$ and target tokens $\mathcal{T}$, ensuring that the transport plan P satisfies the marginal constraints. The vector u and v are dual variables that iteratively scale the source and target tokens, adjusting the transport plan to match the distributions p and q . During each iteration, u and v are updated to

Algorithm 1 Sinkhorn Algorithm for Token Pruning

Inputs: Visual Tokens (Source Tokens) $\mathcal{S}$, Target Tokens $\mathcal{T}$, Visual-aware Cost Matrix $\mathbf{C}_v$, Temperature Parameter ϵ , Max Iteration T , Tolerance δ .

Outputs: Contribution Matrix $\mathbf{P}^*$, Pruned Visual Tokens $\mathcal{S}_p$.

```
1:  $\mathbf{p}, \mathbf{q} \leftarrow \text{Norm}(\mathcal{S}), \text{Norm}(\mathcal{T})$ 
2:  $\mathbf{u}, \mathbf{v} \leftarrow \mathbf{1}^{[N]}, \mathbf{1}^{[N]}$ 
3:  $\mathbf{K} \leftarrow \exp(-\mathbf{C}_v/\epsilon)$ 
4:  $t \leftarrow 0$ 
5: repeat
6:    $t \leftarrow t + 1$ 
7:    $\mathbf{v}_{t+1} \leftarrow \mathbf{q}/(\mathbf{K}^T \exp(\mathbf{u}_t/\epsilon))$ 
8:    $\mathbf{u}_{t+1} \leftarrow \mathbf{p}/(\mathbf{K} \exp(\mathbf{v}_{t+1}/\epsilon))$ 
9: until  $((\|\mathbf{u}_t - \mathbf{u}_{t+1}\| < \delta) \text{ and } (\|\mathbf{v}_t - \mathbf{v}_{t+1}\| < \delta)) \text{ or } t \geq T$ 
10:  $\mathbf{P}^* \leftarrow \mathbf{u}\mathbf{K}\mathbf{v}$  ▷ Contribution Matrix.
11:  $\mathbf{I} \leftarrow \text{Sum}(\mathbf{P}^*, \text{dim} = 1)$  ▷ Token Importance.
12:  $\mathbf{I} \leftarrow \text{Sort}(\mathbf{I})$ 
13:  $\mathcal{S}_{top} \leftarrow \text{TopK}(\mathbf{I})$  ▷ Important Tokens.
14: Remove the unimportant visual tokens  $\mathcal{S}_p = \mathcal{S} \setminus \mathcal{S}_{top}$ 
```

refine the transport plan $\mathbf{P}$, ultimately balancing the flow between source and target tokens according to these distributions. Empirically, the convergence is very fast for our scenarios, which only require three iterations.

Calculating the Importance of Source Token. After solving the problem, we obtain the Contribution Matrix $\mathbf{P}^* \in \mathbb{R}^{N \times N}$ where each element $\mathbf{P}_{ij}^*$ indicates the contribution of source token s_i in constructing target token t_j . Then, we compute Token Importance $\mathbf{I}$ by performing the row-wise summation of $\mathbf{P}^*$, yielding $\mathbf{I}_i = \sum_{j=1}^N \mathbf{P}_{i,j}^*$ for $i = 1, 2, \dots, N$ where $\mathbf{I} \in \mathbb{R}^{N \times 1}$. Each element in $\mathbf{I}$ represents the individual importance of a source token. To perform token pruning, we select the top k tokens with the highest importance scores in $\mathbf{I}$ and discard the remaining less important tokens.

3.5. Token Recovering

Following the pruning process, we introduce an additional uniform sampling-based token recovery method to reintegrate certain tokens in a structured manner, helping to prevent the potential visual tokens collapse, particularly in tasks where the generated tokens need to focus on diverse regions of the input image, such as the OCR task. As shown in Fig. 1, this approach involves uniformly sampling a subset of pruned tokens to restore, maintaining a balanced representation, and reducing the risk of visual collapse.

3.6. Overall Algorithm

The overall pipeline of TopV is shown in Fig. 1. During the prefiling stage, we begin by selecting the input visual

tokens at layer L . We then collect the output tokens after the Post-LN layer within the same layer as target tokens. Next, we formulate and solve the optimization problem to determine token importance. Based on this, we prune the unimportant tokens and selectively recover some pruned tokens in a uniform pattern. Starting from the layer L_{i+1} , the corresponding tokens are consistently pruned, resulting in faster, low-memory VLM inference. We follow [5] to set $L_i = 2$ in our experiments. Notably, our optimization process takes only 2 ms, accounting for less than 1% of total inference time in most cases.

4. Experiments

4.1. Experimental Settings

Model Settings. To demonstrate the effectiveness of our method, we conduct mainly experiments on LLaVA-v1.5-7B and InternVL2-2B. LLaVA-v1.5-7B includes a larger language model with 576 visual input tokens, while InternVL2-2B has a smaller language model with variable visual input tokens from 256 to 1792 due to the dynamic image augmentation. Additionally, we evaluate our method on larger-scale models such as LLaVA-13B and InternVL2-26B.

Evaluation and Datasets. To evaluate the performance in a task-agnostic setting with zero-shot, we conduct the experiments on a variety of VLM tasks, such as recognition and reasoning tasks with AI2D [17], SQA_image [32], MMMU [57], MMBench [30], POPE [21]; image captioning tasks with Npcaps [1] dataset; visual question answering tasks with OK-VQA [34]; and OCR tasks with OCRBench [29].

Implementation Details. The experiments are conducted on A6000 GPU for LLaVA-v1.5-7B and InternVL2-2B model, A100 GPU for LLaVA-v1.5-13B and InternVL2-26B model. The evaluation is performed with the Lmms-Eval [60] library. We set $\sigma = 10$ in Eq. (3). For LLaVA models, we set the hyperparameters α , β , and γ to 1, 1, and 0.01, respectively. For InternVL2 models, we set α , β , and γ to 1, 1, and 0.1, respectively. We set the pruning ratio as 50% and the sampling interval to 4 for token recovering, achieving a 35% FLOPs reduction in overall vision tokens for LLaVA models. With a pruning ratio of 60% and a sampling interval of 6, our setting results in reducing round 50% visual token FLOPs reduction. For InternVL2 models, we set the pruning ratio as 70%, and the sampling interval to 3, resulting in reducing round 47% FLOPs ratio for the visual tokens.

4.2. Main Results

Token Pruning Result on Various Tasks. Tab. 1 and 2 present the performance of the models after applying the TopV framework, with metrics performance score, memory usage (Mem.), and token throughput (Tput.). Notably,

Table 1. Performance comparisons in AI2D, SQA_IMG, MMMU, MMBench tasks. *Score* represents the performance of tasks, *Mem.* indicates maximum memory usage (in GB) for corresponding tasks, and *Tput.* denotes the token throughput, measured in tokens per second, calculated as the total inference time divided by the number of tokens generated. *FLOPs Ratio* indicated the proportion of reduced computation relative to the original vision tokens across all tasks. *Lat.* represents the whole inference latency for corresponding tasks, in minutes and seconds (mm:ss) format. For MMBench, both CN and EN metrics are considered.

Model	Method	FLOPs Ratio ↑	AI2D				SQA_IMG				MMMU				MMBench			
			Score↑	Mem.↓	Lat.↓	Tput.↑	Score↑	Mem.↓	Lat.↓	Tput.↑	Score↑	Mem.↓	Lat.↓	Tput.↑	Score↑	Mem.↓	Lat.↓	Tput.↑
LLaVA-v1.5-7B	Baseline	0	55.18	14.17	10'03	5.13	69.51	14.37	6'33	5.18	35.1	35.90	44'25	5.09	59.97	15.05	77'50	5.29
LLaVA-v1.5-7B	TopV	35%	55.41	13.98	9'25	6.07	69.56	14.3	6'02	6.18	35.4	35.9	41'52	5.76	60.42	14.9	72'36	6.4
LLaVA-v1.5-7B	FastV	47%	55.27	14.69	11'15	4.69	68.91	16.54	7'36	4.47	35.8	45.73	59'58	3.78	59.62	21.14	98'01	4.20
LLaVA-v1.5-7B	TopV	51%	55.31	13.87	8'30	6.12	69.61	14.17	5'32	6.27	35.1	35.45	39'30	5.82	59.65	14.77	67'02	6.46
LLaVA-v1.5-13B	Baseline	0	59.29	28.61	9'43	5.29	72.93	28.92	6'15	5.3	35	49.13	42'51	5.26	65.46	30.06	74'38	5.43
LLaVA-v1.5-13B	TopV	35%	59.38	28.27	8'15	6.25	73.27	28.66	5'22	6.26	35.2	47.79	36'51	6.11	65.63	29.41	63'58	6.32
LLaVA-v1.5-13B	FastV	48%	58.91	29.39	10'09	5.07	73.12	29.51	7'02	4.78	34.3	75.32	58'58	3.83	64.95	40.12	86'48	4.63
LLaVA-v1.5-13B	TopV	50%	59.27	27.91	7'58	6.45	73.17	28.32	5'07	6.57	34.7	47.29	35'21	6.38	65.16	29.11	60'23	6.56
InternVL2-2B	Baseline	0	72.67	6.39	9'10	5.61	94.2	6.63	6'07	5.5	34.56	13.72	39'57	5.25	67.6	6.78	86'57	12.1
InternVL2-2B	FastV	47%	71.57	6.92	11'16	4.64	93.75	7.69	7'45	4.34	33.67	32.65	52'46	3.98	69.46	7.62	136'56	7.68
InternVL2-2B	TopV	48%	73.35	5.57	7'50	6.57	94.22	5.99	4'50	6.39	34.6	13.33	36'20	5.93	69.7	5.98	75'17	14.57
InternVL2-26B	Baseline	0	83.13	56.79	41'57	2.46	97.47	57.61	28'43	2.34	47.11	69.02	173'47	2.24	80.89	57.02	342'27	4.24
InternVL2-26B	FastV	46%	81.27	57.67	49'05	2.1	96.82	58.12	33'12	2.02	46.91	165.23	230'12	1.69	80.86	58.87	490'38	2.96
InternVL2-26B	TopV	47%	83.34	55.91	34'03	2.86	97.67	56.73	24'26	2.75	46.98	68.26	157'36	2.47	81.27	56.33	300'52	4.83

Table 2. Performance comparisons in POPE, Nocaps, OCRBench, OK-VQA tasks.

Model	Method	FLOPs Ratio ↑	POPE				Nocaps(CIDEr)				OCRBench				OK-VQA			
			Score↑	Mem.↓	Lat.↓	Tput.↑	Score↑	Mem.↓	Lat.↓	Tput.↑	Score↑	Mem.↓	Lat.↓	Tput.↑	Score↑	Mem.↓	Lat.↓	Tput.↑
LLaVA-v1.5-7B	Baseline	0	85.81	13.96	25'59	5.77	105.67	13.93	172'30	22.10	31.3	14.00	15'05	22.67	53.38	13.98	19'42	9.54
LLaVA-v1.5-7B	TopV	35%	84.96	13.78	24'27	6.54	104.12	13.78	165'39	25.77	31.6	13.89	14'36	24.96	52.96	13.81	18'37	10.91
LLaVA-v1.5-7B	FastV	47%	82.21	14.10	29'09	5.15	104.08	14.07	351'08	10.86	30.3	14.45	31'20	10.91	52.87	14.04	26'08	7.19
LLaVA-v1.5-7B	TopV	51%	84.21	13.7	22'16	6.59	103.52	13.7	158'01	25.81	31	13.77	14'03	25.01	52.64	13.76	17'20	10.97
LLaVA-v1.5-13B	Baseline	0	86.01	28.53	24'26	6.14	109.37	28.79	160'32	22.98	33.6	28.97	14'34	24.29	58.2	28.39	18'59	10.08
LLaVA-v1.5-13B	TopV	35%	86.12	28.07	21'31	7.08	108.23	28.32	144'28	26.15	34.1	28.39	13'03	27.62	58.02	27.91	17'12	10.99
LLaVA-v1.5-13B	FastV	48%	84.96	29.02	28'56	5.18	107.71	29.37	314'29	12.12	32.9	29.57	31'06	10.99	58.02	28.89	25'19	7.43
LLaVA-v1.5-13B	TopV	50%	85.53	27.79	20'30	7.32	107.67	28.02	138'52	27.61	33.6	28.13	12'26	28.97	57.92	27.63	16'46	11.27
InternVL2-2B	Baseline	0	88.39	6.25	33'05	4.53	60.34	6.23	150'10	30.41	75.6	6.26	8'30	30.83	43.59	6.28	20'24	9.46
InternVL2-2B	FastV	47%	87.89	6.45	41'05	3.65	60.22	6.58	432'57	10.55	68.8	6.78	22'38	12.03	43.28	6.56	31'02	6.22
InternVL2-2B	TopV	48%	88.02	5.68	25'05	5.55	64.16	5.45	130'20	32.17	72.9	5.85	7'39	34.79	43.02	5.47	18'17	10.79
InternVL2-26B	Baseline	0	87.98	56.63	157'40	1.92	92.76	56.74	371'01	8.98	77.9	56.91	23'43	12.18	48.41	56.93	89'48	3.19
InternVL2-26B	FastV	46%	87.06	57.36	194'29	1.56	93.62	57.93	910'12	3.66	73.5	58.21	50'02	5.78	48.23	58.17	135'02	2.12
InternVL2-26B	TopV	47%	87.32	56.02	128'54	2.35	95.02	56.11	320'29	10.42	75.1	56.22	20'10	14.33	48.12	56.08	81'21	3.53

memory usage is divided into static and dynamic components. Static memory refers to the memory used for model parameters, which remains unaffected by token pruning. For instance, the static memory usage of LLaVA-v1.5-7B and InternVL2-2B is approximately 13.8GB and 4.5GB, respectively. Dynamic memory, on the other hand, represents the runtime memory increase and more accurately reflects the effectiveness of various token pruning methods. In addition to the overall memory usage reported in Tab. 1, we further analyze dynamic memory usage in our comparisons.

For the InternVL2 models, when reducing FLOPs by round 47% for the vision tokens, the model showed almost no loss of accuracy compared to the baseline, while dynamic memory usage is reduced by 23.8% and inference efficiency improved to $1.23 \times$ on average and improved to $1.31 \times$ for some recognition tasks. When compared to FastV, which reduces FLOPs by 47%, InternVL2 models

shows a 1.2 higher score on average, with dynamic memory usage decreasing by 61% and inference efficiency improving by $2.1 \times$.

For the LLaVA models, reducing FLOPs by 35% of the vision tokens results in negligible accuracy loss compared to the baseline, with inference efficiency improving to $1.08 \times$ on average and improving to $1.12 \times$ for some recognition tasks. When reducing FLOPs round 50% of the vision tokens, the model outperforms the FastV, which reduces FLOPs by 47%, by an average of 0.39% in accuracy. Additionally, dynamic memory usage is reduced by 49%, and inference efficiency improves by $1.68 \times$.

Memory Usage. To further investigate the impact on memory usage resulting from compatibility with the KV cache, we conduct detailed profiling of memory usage on AI2D and OCRBench during the inference process. we select a few random samples for each task after GPU warmup (Fig.

Table 3. Performance comparison with VTW in AI2D, SQA_IMG, MMMU, MMBench, POPE, Nocaps, OCRBench, OK-VQA tasks.

Model	Method	FLOPs Ratio	AI2D	SQA_IMG	MMMU	MMBench	POPE	Nocaps	OCRBench	OK-VQA
LLaVA-v1.5-7B	VTW	50%	55.41	69.56	36.3	59.63	85.92	58.37	5.1	39.88
LLaVA-v1.5-7B	TopV	51%	55.31	69.61	35.1	59.65	84.21	103.52	31	52.64

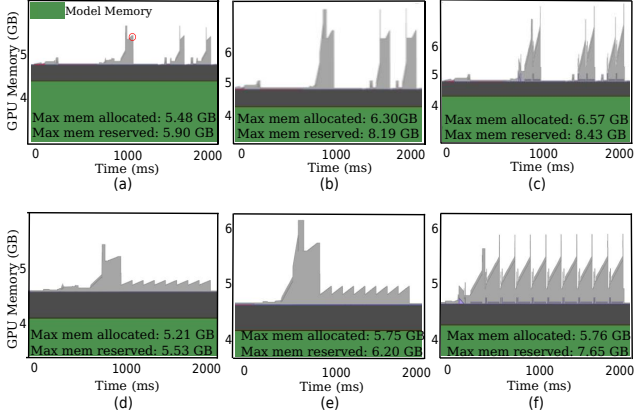


Figure 3. Memory Usage of TopV (a, d), Baseline (b, e), and FastV (c, f) on AI2D and OCRBench tasks for InternVL2-2B. The red circle indicates that a new token is currently being processed.

3). Subfigures (a), (b), and (c) represent the memory usage of TopV, baseline, and FastV on the AI2D task, respectively, and (d), (e), (f) show the memory usage of these methods on the OCRBench task. In AI2D, a recognition task, memory is primarily allocated for the answer and end-of-text tokens (highlighted in Fig. 3 (a)). With KV cache, memory from the first token is efficiently reused for the second, preventing extra allocations. In contrast, without KV cache (Fig. 3 (c)), memory usage spikes again when generating the second token, indicating inefficient reuse. For OCR tasks, which generate multiple tokens, the absence of KV cache leads to continuous memory peaks, as each token requires additional allocation, increasing overall memory consumption.

Based on these observations, we assume that, although some intermediate values (e.g., tokens in VLMs) may not need to be retained during inference to save memory, when the remaining GPU memory is sufficient and the retained memory is not efficiently reusable, the CUDA backend opts for faster memory allocation strategies. This involves re-allocating memory, rather than reusing the currently freed memory and then allocating new memory. It is important to note that we are discussing this issue in the context of a more general, off-the-shelf approach.

Comparison with Vision Tokens Withdrawal in Middle Layer. As shown in Tab. 3, we also compared our method with the Vision Token Withdrawly [25] approach, under the condition of reducing an equivalent proportion of vision tokens involved in computation. Unlike the token pruning method, VTW begins withdrawing all vision tokens from

Table 4. Impact of different target token positions within a block of the LLaVA-v1.5-7B model. Positions 1, 2, 3, and 4 represent the outputs from the Pre-LN layer, Attention layer, Post-LN layer, and MLP layer, respectively, as illustrated in Fig. 2. The "2*" represents the outputs from position 2 without the use of the residual module.

Position	AI2D	SQA_IMG	OK-VQA
Baseline	55.18	69.51	53.38
1	54.17	68.32	47.28
2*	54.22	68.37	50.22
2	54.27	68.86	48.99
3	54.99	69.16	51.55
4	53.21	66.41	43.98

Table 5. Evaluation Results on Video Question Answering Tasks.

Model	Method	FLOPs Ratio ↓	TGIF		VideoMME(Overall)	
			Acc↑	Lat.↓	Acc↑	Lat.↓
Video-LLaVA-7B	Baseline	0	0.27	414'13	0.3	41'20
Video-LLaVA-7B	FastV	47%	0.24	670'30	0.29	67'11
Video-LLaVA-7B	TopV	51%	0.25	370'23	0.3	35'52

a block in the middle of the LLM. While this method is effective for tasks such as recognition questions, it performs poorly on tasks that require generating a large number of tokens, such as VQA, Caption, and OCR tasks. The reason is that, as the generated tokens evolve, the required vision features also change. However, in VTW's approach, once tokens are withdrawn from the middle, no vision tokens remain, leading to a loss of crucial vision information.

Comparison with FastV on Video Understanding Tasks. As shown in Tab. 5, we compared our method with the FastV approach on video Understanding tasks. The results show that our method can outperform the FastV even with the higher visual token FLOPs reduction ratio.

4.3. Ablation Studies

Impact of Outputs from Different Layers as Target Tokens. As discussed in Sec 3, if a layer chosen for output is too early in the entire block, its output may not adequately represent the function of the current block. On the other hand, if the selected layer is too late, the presence of two residual connections may cause the output to be too similar to the source tokens. Here, we conduct the ablation studies only with the feature similarity factor. As shown in Tab. 4, selecting the output from Post-LN (3), as the target yields the best results. In contrast, using the MLP output as the output produces the poorest performance.

Impact of Token Recovering. To evaluate the effectiveness of our token recovery method, we conducted experi-

Table 6. Effect of Token Recovering. The results show the token pruning with and without the proposed token recovery method.

Model	Method	Sampling	OCRBench
LLaVA-v1.5-7B	Baseline	-	31.3
LLaVA-v1.5-7B	TopV	w/o	29.7
LLaVA-v1.5-7B	TopV	w/	30.5
InternVL2-2B	Baseline	-	75.6
InternVL2-2B	TopV	w/o	70.5
InternVL2-2B	TopV	w/	71.3

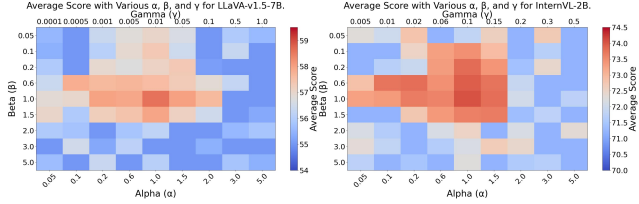


Figure 4. Two visualization heat maps for various hyperparameters.

ments to measure the impact on different tasks with Feature Similarity Cost. To achieve the FLOPs reductions of 35% in LLaVA-v1.5-7B, we set the pruning ratio to 35% when no token recovery is applied. When the token recovery is employed, the pruning ratio is set to 50%. As shown in Table 6, token recovering has clear advantages in tasks that require generating a layer number of tokens to capture potentially useful tokens in subsequent steps.

Impact of Different Components of Visual-Aware Cost Function. To validate the impact of each cost function in the Visual-Aware Cost Function with token recovering, we conducted experiments on four tasks. As shown in Tab. 7, the different cost functions are compatible and work together to enhance performance.

The Sensitive to Various Hyperparameters. With various α, β, γ , we conduct the experiments and obtain the average score on AI2D, SQA_IMG, POPE, OCRBench, and OK-VQA. As shown in Fig. 4, the heatmaps for LLaVA-v1.5-7B (left) and InternVL2-2B (right) illustrate that while various hyperparameters affect performance, adjacent hyperparameter values yield similar scores, indicating that our method is not highly sensitive to them.

4.4. Visualization

Important Token Distribution from various Pruning Method. The important vision tokens selected by various pruning methods are visualized as shown in Fig. 5. Compared to FastV, TopV concentrates more tokens in the foreground region with red patches, avoiding unnecessary focus on the background or padded areas with gray patches, especially for surrounding padding areas.

Table 7. Impact of Different Components in Visual-Aware Cost Function. The 'F', 'S', and 'E' represent Feature Similarity, Relative Spatial Distance, and Absolute Distance Cost, respectively.

Model	Method	Costs	AI2D	SQA_IMG	MMMU	OK-VQA
LLaVA-v1.5-7B	Baseline	-	55.18	69.51	35.1	53.38
LLaVA-v1.5-7B	TopV	F	54.99	69.16	35.1	51.55
LLaVA-v1.5-7B	TopV	F+S	55.38	69.61	35.3	52.86
LLaVA-v1.5-7B	TopV	F+S+E	55.41	69.56	35.4	52.96
InternVL2-2B	Baseline	-	72.67	94.2	34.56	43.59
InternVL2-2B	TopV	F	72.28	93.82	33.36	41.55
InternVL2-2B	TopV	F+S	73.32	94.12	34.36	42.97
InternVL2-2B	TopV	F+S+E	73.35	94.22	34.6	43.02



Figure 5. Two visualization examples of important tokens identified by various visual token pruning methods. **Left:** Important tokens selected by FastV. **Right:** Important tokens selected by our TopV. The gray patches around the images are introduced during preprocessing, while the red patches indicate the token regions the model focuses on.

5. Conclusion

In this paper, we present TopV, a novel token pruning method designed to optimize inference efficiency in VLMs without additional training or fine-tuning. By formulating token pruning as an optimization problem, TopV effectively identifies and prunes redundant visual tokens, reducing computational demand and memory requirements. Our approach maintains compatibility with FlashAttention and reduces KV cache storage by performing pruning only once during the prefilling stage. Extensive experiments demonstrate that our method outperforms previous methods, validating the effectiveness and efficiency of our approach.

Acknowledgment This work was supported in part by the NSF through award 2217154, NSF IIS-2224843, ITE-2429680, CCF-2239945, and CMMI-2152908.

References

- [1] Harsh Agrawal, Karan Desai, Yufei Wang, Xinlei Chen, Rishabh Jain, Mark Johnson, Dhruv Batra, Devi Parikh, Stefan Lee, and Peter Anderson. Nocaps: Novel object captioning at scale. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 8948–8957, 2019. 5
- [2] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. Vqa: Visual question answering. In *Proceedings of the IEEE international conference on computer vision*, pages 2425–2433, 2015. 1
- [3] Florian Bordes, Richard Yuanzhe Pang, Anurag Ajay, Alexander C Li, Adrien Bardes, Suzanne Petryk, Oscar Mañas, Zhiqiu Lin, Anas Mahmoud, Bargav Jayaraman, et al. An introduction to vision-language modeling. *arXiv preprint arXiv:2405.17247*, 2024. 1
- [4] Jieneng Chen, Luoxin Ye, Ju He, Zhao-Yang Wang, Daniel Khashabi, and Alan Yuille. Llavolta: Efficient multi-modal models via stage-wise visual context compression. *arXiv preprint arXiv:2406.20092*, 2024. 2
- [5] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*, pages 19–35, 2024. 1, 2, 3, 5
- [6] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, Jiapeng Luo, Zheng Ma, et al. How far are we to gpt-4v? closing the gap to commercial multimodal models with open-source suites. *arXiv preprint arXiv:2404.16821*, 2024. 1
- [7] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 24185–24198, 2024. 1
- [8] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. *Advances in neural information processing systems*, 26, 2013. 4
- [9] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022. 1, 2
- [10] Jacob Devlin, Hao Cheng, Hao Fang, Saurabh Gupta, Li Deng, Xiaodong He, Geoffrey Zweig, and Margaret Mitchell. Language models for image captioning: The quirks and what works. *arXiv preprint arXiv:1505.01809*, 2015. 1
- [11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. 4
- [12] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. 1
- [13] Masato Fujitake. Dtrocr: Decoder-only transformer for optical character recognition. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 8025–8035, 2024. 1
- [14] Zhangwei Gao, Zhe Chen, Erfei Cui, Yiming Ren, Weiyun Wang, Jinguo Zhu, Hao Tian, Shenglong Ye, Junjun He, Xizhou Zhu, et al. Mini-internvl: A flexible-transfer pocket multimodal model with 5% parameters and 90% performance. *arXiv preprint arXiv:2410.16261*, 2024. 1
- [15] Aude Genevay, Marco Cuturi, Gabriel Peyré, and Francis Bach. Stochastic optimization for large-scale optimal transport. *Advances in neural information processing systems*, 29, 2016. 3
- [16] Yunseok Jang, Yale Song, Youngjae Yu, Youngjin Kim, and Gunhee Kim. Tgif-qa: Toward spatio-temporal reasoning in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2758–2766, 2017. 1
- [17] Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14*, pages 235–251. Springer, 2016. 5
- [18] Philip A Knight. The sinkhorn–knopp algorithm: convergence and applications. *SIAM Journal on Matrix Analysis and Applications*, 30(1):261–275, 2008. 4
- [19] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023. 2
- [20] Minghao Li, Tengchao Lv, Jingye Chen, Lei Cui, Yijuan Lu, Dinei Florencio, Cha Zhang, Zhoujun Li, and Furu Wei. Trocr: Transformer-based optical character recognition with pre-trained models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 13094–13102, 2023. 1
- [21] Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. *arXiv preprint arXiv:2305.10355*, 2023. 5
- [22] Yanwei Li, Chengyao Wang, and Jiaya Jia. Llama-vid: An image is worth 2 tokens in large language models. In *European Conference on Computer Vision*, pages 323–340. Springer, 2025. 2
- [23] Bin Lin, Yang Ye, Bin Zhu, Jiaxi Cui, Munan Ning, Peng Jin, and Li Yuan. Video-llava: Learning united visual representation by alignment before projection. *arXiv preprint arXiv:2311.10122*, 2023. 2
- [24] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 2024. 1

- [25] Zhihang Lin, Mingbao Lin, Luxi Lin, and Rongrong Ji. Boosting multimodal large language models with visual tokens withdrawal for rapid inference. *arXiv preprint arXiv:2405.05803*, 2024. 1, 2, 7
- [26] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26296–26306, 2024. 2
- [27] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, 2024. 2
- [28] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36, 2024. 1, 2
- [29] Yuliang Liu, Zhang Li, Biao Yang, Chunyuan Li, Xucheng Yin, Cheng-lin Liu, Lianwen Jin, and Xiang Bai. On the hidden mystery of ocr in large multimodal models. *arXiv preprint arXiv:2305.07895*, 2023. 5
- [30] Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, et al. Mmbench: Is your multi-modal model an all-around player? In *European Conference on Computer Vision*, pages 216–233. Springer, 2025. 5
- [31] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024. 1
- [32] Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. *Advances in Neural Information Processing Systems*, 35:2507–2521, 2022. 5
- [33] Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. Video-chatgpt: Towards detailed video understanding via large vision and language models. *arXiv preprint arXiv:2306.05424*, 2023. 1
- [34] Kenneth Marino, Mohammad Rastegari, Ali Farhadi, and Roozbeh Mottaghi. Ok-vqa: A visual question answering benchmark requiring external knowledge. In *Proceedings of the IEEE/cvf conference on computer vision and pattern recognition*, pages 3195–3204, 2019. 5
- [35] Sachit Menon and Carl Vondrick. Visual classification via description from large language models. *arXiv preprint arXiv:2210.07183*, 2022. 1
- [36] Ron Mokady, Amir Hertz, and Amit H Bermano. Clip-cap: Clip prefix for image captioning. *arXiv preprint arXiv:2111.09734*, 2021. 1
- [37] Eduardo Fernandes Montesuma, Fred Maurice Ngolè Mboula, and Antoine Souloumiac. Recent advances in optimal transport for machine learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 3
- [38] Shunji Mori, Hirobumi Nishida, and Hiromitsu Yamada. *Optical character recognition*. John Wiley & Sons, Inc., 1999. 1
- [39] Gabriel Peyré, Marco Cuturi, et al. Computational optimal transport: With applications to data science. *Foundations and Trends® in Machine Learning*, 11(5-6):355–607, 2019. 3
- [40] Khiem Pham, Khang Le, Nhat Ho, Tung Pham, and Hung Bui. On unbalanced optimal transport: An analysis of sinkhorn algorithm. In *International Conference on Machine Learning*, pages 7673–7682. PMLR, 2020. 4
- [41] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021. 2, 4
- [42] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. Llava-prumerge: Adaptive token reduction for efficient large multimodal models. *arXiv preprint arXiv:2403.15388*, 2024. 1, 2, 4
- [43] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. 4
- [44] Yang Sui, Miao Yin, Yi Xie, Huy Phan, Saman Aliari Zonouz, and Bo Yuan. Chip: Channel independence-based pruning for compact neural networks. *Advances in Neural Information Processing Systems*, 34:24604–24616, 2021. 1
- [45] Yang Sui, Yanyu Li, Anil Kag, Yerlan Idelbayev, Junli Cao, Ju Hu, Dhritiman Sagar, Bo Yuan, Sergey Tulyakov, and Jian Ren. Bitsfusion: 1.99 bits weight quantization of diffusion model. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [46] Yang Sui, Miao Yin, Yu Gong, and Bo Yuan. Co-exploring structured sparsification and low-rank tensor decomposition for compact dnns. *IEEE Transactions on Neural Networks and Learning Systems*, 2024. 1
- [47] Keda Tao, Can Qin, Haoxuan You, Yang Sui, and Huan Wang. Dycok: Dynamic compression of tokens for fast video large language models. *arXiv preprint arXiv:2411.15024*, 2024. 1
- [48] Cédric Villani et al. *Optimal transport: old and new*. Springer, 2009. 3
- [49] Wenhao Wu, Zhun Sun, Yuxin Song, Jingdong Wang, and Wanli Ouyang. Transferring vision-language models for visual recognition: A classifier perspective. *International Journal of Computer Vision*, 132(2):392–409, 2024. 1
- [50] Jinqi Xiao, Miao Yin, Yu Gong, Xiao Zang, Jian Ren, and Bo Yuan. Comcat: towards efficient compression and customization of attention-based vision models. *arXiv preprint arXiv:2305.17235*, 2023. 1
- [51] Jinqi Xiao, Chengming Zhang, Yu Gong, Miao Yin, Yang Sui, Lizhi Xiang, Dingwen Tao, and Bo Yuan. Haloc: hardware-aware automatic low-rank compression for compact neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 10464–10472, 2023.
- [52] Jinqi Xiao, Shen Sang, Tiancheng Zhi, Jing Liu, Qing Yan, Linjie Luo, and Bo Yuan. Coap: Memory-efficient training with correlation-aware gradient projection. *arXiv preprint arXiv:2412.00071*, 2024. 1

- [53] Dejing Xu, Zhou Zhao, Jun Xiao, Fei Wu, Hanwang Zhang, Xiangnan He, and Yueting Zhuang. Video question answering via gradually refined attention over appearance and motion. In *Proceedings of the 25th ACM international conference on Multimedia*, pages 1645–1653, 2017. [1](#)
- [54] Cheng Yang, Yang Sui, Jinqi Xiao, Lingyi Huang, Yu Gong, Yuanlin Duan, Wenqi Jia, Miao Yin, Yu Cheng, and Bo Yuan. Moe-i²: Compressing mixture of experts models through inter-expert pruning and intra-expert low-rank decomposition. *arXiv preprint arXiv:2411.01016*, 2024. [1](#)
- [55] Linli Yao, Lei Li, Shuhuai Ren, Lean Wang, Yuanxin Liu, Xu Sun, and Lu Hou. Deco: Decoupling token compression from semantic abstraction in multimodal large language models. *arXiv preprint arXiv:2405.20985*, 2024. [2](#)
- [56] Li Yuan, Qibin Hou, Zihang Jiang, Jiashi Feng, and Shuicheng Yan. Volo: Vision outlooker for visual recognition. *IEEE transactions on pattern analysis and machine intelligence*, 45(5):6575–6586, 2022. [1](#)
- [57] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruochi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9556–9567, 2024. [5](#)
- [58] Hang Zhang, Xin Li, and Lidong Bing. Video-llama: An instruction-tuned audio-visual language model for video understanding. *arXiv preprint arXiv:2306.02858*, 2023. [2](#)
- [59] Jingyi Zhang, Jiaying Huang, Sheng Jin, and Shijian Lu. Vision-language models for vision tasks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. [1](#)
- [60] Kaichen Zhang, Bo Li, Peiyuan Zhang, Fanyi Pu, Joshua Adrian Cahyono, Kairui Hu, Shuai Liu, Yuanhan Zhang, Jingkang Yang, Chunyuan Li, et al. Lmms-eval: Reality check on the evaluation of large multimodal models. *arXiv preprint arXiv:2407.12772*, 2024. [5](#)
- [61] Pengchuan Zhang, Xiujun Li, Xiaowei Hu, Jianwei Yang, Lei Zhang, Lijuan Wang, Yejin Choi, and Jianfeng Gao. Vinvl: Revisiting visual representations in vision-language models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5579–5588, 2021. [1](#)
- [62] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36: 34661–34710, 2023. [1](#)
- [63] Luwei Zhou, Hamid Palangi, Lei Zhang, Houdong Hu, Jason Corso, and Jianfeng Gao. Unified vision-language pre-training for image captioning and vqa. In *Proceedings of the AAAI conference on artificial intelligence*, pages 13041–13049, 2020. [1](#)